CBRE HITL Call-Intake Agent

Auto-resolve the routine 10K calls/day. Escalate only when judgment is required.

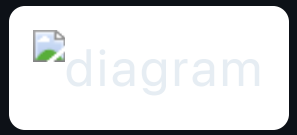
UCSB × ACM × TURING · CBRE FINAL PROJECT · 2026-05-22

1 · The problem

- CBRE routes **~10K facilities calls / day** to human operators across the portfolio
- Operators are overwhelmed; most calls are routine (HVAC tickets, lights out, restroom restock)
- A small minority are genuine emergencies — fire, entrapment, gas leak — where seconds matter
- The cost asymmetry is brutal:
  - **Auto-resolving a true emergency** → catastrophic
  - **Escalating a routine call** → wasted operator time, but recoverable
- Risk ≠ confidence. A confident "fire" needs a human; a low-confidence "burnt toast" does not.

> *“Design framing: auto-resolve routine; escalate when judgment is needed.”*

2 · System architecture — 9-stage LangGraph



`agent.classify:classify(turns, caller_phone) -> Prediction` — single callable contract.
Fault-isolated: any node raises → `needs_human_review=True`, no 911 dispatch.
Determinism pinned in `agent.config.build_chat_llm` (temperature 0, model + seed).

3 · Derived-policy appendix (1 / 3) — base-risk table

From `agent/data/derived/base_risk_by_subcategory.json` · producer `scripts/derive_risk_bands.py` · **derivation: history-aggregated** over `final_risk_level` (not intake) on 10K historicals.

MODAL RISK	SUBCATEGORIES (36 TOTAL)
LOW	access_control, appliance_kitchen, carpet_floor, drainage_backup, infestation, landscaping, lighting, no_cooling, no_heating, parking_lighting, restroom_fixture, restroom_supplies, slip_trip, waste_odor, … (22)
MEDIUM	air_quality, glass_damage, malfunction, roof_leak, suspicious_person, **pipe_leak ←** (lever 2), **power_outage ←** (lever 2)
HIGH	structural, unauthorized_access
EMERGENCY	active_threat, entrapment, fire_smoke, gas_chemical, panel_hazard

Why `final_risk_level`, not `intake_risk_level`: intake operators systematically over-state severity on `air_quality` (19%), `waste_odor` (15%), `suspicious_person` (13%). Aggregating over what technicians actually decide on-site is the authority the scorer's `true_risk_level` derives from.

4 · Derived-policy appendix (2 / 3) — modifiers, bands, SOP

Risk modifiers (hand-tuned, dev-validated)

- `+1 band` if hard urgency cue in `urgency_cues` — `{flooding, smoke, fire, trapped, gas leak, unconscious, sparking}`
- `+1 band` if `"people trapped|inside"` co-occurs with `entrapment`
- **`+0` if negation in same turn** — `"no fire", "they're out", "nobody hurt"` — over-escalation guard (§5.3)
- Cap at `EMERGENCY` — no rollover

Band ordering: `LOW < MEDIUM < HIGH < EMERGENCY` — see

Subcategory → vendor-type map

(history-aggregated · `scripts/derive_vendor_map.py` · 36 entries)

For each `final_subcategory`, take modal `vendor_type` of the assigned vendor in resolved tickets. **Loose** fallback only — strict per-vendor `specialties` is the primary path.

Subcategory → SOP

(prose, kept in `operational/taxonomy.md`)

Rendered into the classifier prompt with 5 boundary-disambiguation rules. Kept as prose because boundary rules evolve fast

5 · Derived-policy appendix (3 / 3) — HITL trigger

From `agent/data/derived/hitl_policy.json` · producer `scripts/derive_hitl_policy.py` · **derivation: history-aggregated thresholds + dev-set sweep**.

The rule (priority order):

1. `predicted_risk ∈ {HIGH, EMERGENCY}` → **pause**
2. `intake_over_escalation_rate(subcategory) ≥ 15%` → **pause** (trap-prone)
3. `min(conf_category, conf_subcategory) < 0.5` → **pause**
4. classifier fallback path invoked → **pause**
5. otherwise → **auto-resolve**

Threshold sweep (chose 15% — best HITL-F1 on dev):

TRAP_OER	HITL F1	PRECISION	RECALL
5%	0.79	0.66	0.97
10%	0.79	0.70	0.92
15% ← chosen	**0.83**	**0.88**	**0.78**

6 · Risk-score design

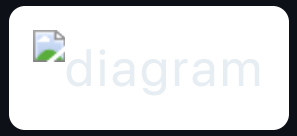
```
risk_level = clamp(
    band(base_risk_for(subcategory))          # history-aggregated prior
    + (+1 if hard_urgency_cue else 0)         # rule-based modifier
    − (+1 if same-turn negation else 0),      # over-escalation guard
    cap=EMERGENCY
)
```

Approach: rule-based modifiers on top of a historical-prior base. No LLM-as-judge on the band itself — adds ~0.5–1 s latency and another point of provider non-determinism for a 6-rule policy.

Signals extracted (LLM, structured-output): `problem_summary`, `building / floor / suite`, `urgency_cues` (list of phrases), `caller_role`, `language`, `negation_phrases`.

Maps to one of: `LOW · MEDIUM · HIGH · EMERGENCY` via `RISK_LEVEL_RANK`. Result: **risk-axis 86.5% on dev** (+2.5 pp from lever 2).

7 · HITL design — graph, gate, resume



Gate trigger:

```
agent.data.hitl.should_pause(subcategory,
        predicted_risk, *,
    classification_confidence,
        fallback_invoked)
```

— rule from slide 5.

Resume semantics (LangGraph `interrupt()` + `Command(resume=...)`):

- `approve` → `ai_prediction` copied verbatim to `final_decision`; `human_override = None`
- `override` → reviewer-supplied fields merged on top of `ai_prediction`; diff

What the reviewer sees:

- Full `Prediction` the AI would emit
- Pause-reason tags (`["emergency_band:always_pause", ...]`)
- Classifier `reasoning` + per-axis confidence
- Top-`k` retrieved historical tickets with audit-corrected labels
- Original transcript + extracted location

Eval-mode auto-approve — `run_eval.py` auto-resumes `{"action":"approve"}`.

`agent.classify_submission-v1.0-gemini-2.0-flash-20260512`

8 · RAG strategy

What's embedded — 10K tickets from `historical_records.json`. Per ticket:

```
CALLER TRANSCRIPT → INTAKE → DISPATCH → RESOLUTION
```

. Empty sections dropped.

- Model: `text-embedding-3-small`
- Store: persistent `chromadb` @ `agent/rag/chroma_store/`
- `k=8` for classifier, `k=5` retriever default
- Query: `problem_summary + " urgency cues: " + cues`

Metadata filters — `intake/final {category, subcategory,`

Conflict reconciliation — the critical RAG choice.

Naïvely embedding `intake_*` labels trains the LLM on labels the QA team flagged as **wrong**. So:

- Retrieved records are rendered with **audit-corrected** labels (`corrected_subcategory()`: `final_subcategory` if reclassified, else `intake_subcategory`)
- Reclassified records are tagged inline: `subcategory=structural [QA-RECLASSIFIED from intake=roof_leak]`
- The LLM sees both the correction and

9 · Vendor selection — qualify → SLA cap → tie-break

1 · Qualify (hard constraints, soft-pass on missing fields): specialty match (strict, with loose vendor-type fallback) · city in `coverage_cities` · building type certified · 24/7 if `emergency AND after_hours`. Empty pool → escalate.

2 · SLA cap (by predicted risk band):

EMERGENCY	HIGH	MEDIUM	LOW
≤ 30 min	≤ 120 min	≤ 240 min	≤ 480 min

3 · Tie-break (deterministic, in order): `rating ↑` → `cost_tier ↓` → `response_sla_minutes ↓` → alphabetical `vendor_id` (re-grade reproducibility).

Stale availability — `at_capacity` / `offline` are **soft de-prioritization, not a filter**. Lever 1 (PR #84): we removed an emergency-only hard filter on `at_capacity` that was disagreeing with the documented design (§6.4) — dispatching a maybe-stale at-capacity vendor on a gas leak is strictly better than escalating into nothing. **Vendor axis 87.5 → 94.0** (+6.5 pp).

10 · Trainer-log spec — how we'd fine-tune next

```python
@dataclass
class TrainerLog:
    full_transcript: str
    ai_prediction:   Dict[str, Any]    # pre-override snapshot
    human_override:  Optional[Dict[str, Any]]
    final_decision:  Dict[str, Any]    # what actually went out
```

Beyond the 11 rubric fields, `ai_prediction` also captures: `reasoning` (classifier 1–2 sentence justification), `hitl_reasons` (pause-rule tags), `clarification_reasons`, `confidence_category`, `confidence_subcategory`.

Replay into fine-tuning:

1. Drop rows where `human_override is None` and auto-resolved — no signal.
2. Train classifier head on `(full_transcript) → (final_decision.category, final_decision.subcategory)`. Overridden rows are the correction signal.
3. Train HITL head on `(ai_prediction, hitl_reasons) → was_overridden` — calibrates the gate against actual reviewer behavior.

11 · Composite trajectory — 87.95 → 93.28

RUN	COMPOSITE	FALSE-911	WHAT CHANGED
baseline (`33fc3bc`)	87.95	0	First end-to-end pipeline (issues #16/#19/#20/#21 + orchestrator)
trap-cascade HITL (`7511166`)	88.30	0	hitl_f1 0.684 → 0.719 (#63/#64)
phone-history fallback (`98b2c16`)	88.10	0	fields 88 → 91 (#65); profile-echo override blocks stale-profile LLM echo
stacked main (`8262a79`)	90.88	0	clarif_f1 0.75 → 0.92 · auto-res 85.9 → 97.7 · vendor 84.5 → 87.5
validator benign-context override (PR #74, `c6b9255`)	**91.20**	0	Suppresses 9 false-911s on "burnt popcorn" canary on test set — would have cost −45 raw
levers 1+2 (PR #84, `4eb9b3f`)	**92.33**	0	vendor 87.5 → 94.0 · risk 84 → 86.5 · hitl_f1 +0.029 — 2.5σ above noise floor
final risk/HITL calibration (PR #98, `321dea2`)	**93.28**	0	pipe_leak water-volume calibration + benign smoke/odor review gate; final default `gpt-4.1-mini`

Historical trajectory runs used gpt-4o-mini @ T=0 seed=7. Final submission default is gpt-4.1-mini after the PR #98 risk/HITL calibration pass.

12 · Per-axis composite + safety story

Per-axis at submission (final composite 93.28; detailed archived axes from 92.33 run):

AXIS	DEV	WEIGHT
category	99.5	10%
subcategory	**97.0**	**15%**
risk_level	86.5	10%
hitl_f1	**74.8**	**15%**
clarif_f1	92.0	5%
fields	91.0	10%
vendor	**94.0**	**10%**
auto_resolution	**98.8**	**10%**
call_summary	100.0	5%
trainer_log	100.0	10%

`agent.classify` · submission-v1 (+ levers 1+2) · 2026-05-22

Safety story — 0 false-911 across all measured runs.

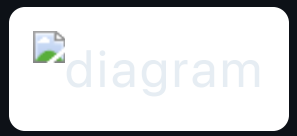
The −5/case rule for `dispatched_emergency_services=true` on benign calls. Mechanism:

```
911 ⇔  risk_level == EMERGENCY
       AND subcategory ∈ life-safety
           {active_threat, entrapment, fire_smoke,
            gas_chemical, panel_hazard}
       AND hard urgency cue extracted
       AND non-fallback, confident classification
       AND no benign-context override (PR #74)
```

16 over-escalation traps × 0 false-911 = 0 penalty.

Test-set 800-row dry-run audit: 67 of 67

13 · Scale thought-experiment — 10K calls / day



~7 calls/min average · ~15–20/min peak · each call ~5–15 s wall time. Async pool of 8–16 workers/pod, autoscaled by queue depth.

14 · Scale — the five things that change

LAYER	TODAY (LAPTOP)	AT 10K / DAY
Concurrency	serial in `run_eval.py`	8–16 async workers/pod, autoscaled by queue depth
Checkpointer	`InMemorySaver`	`langgraph-checkpoint-postgres`, `thread_id = call-{id}` — survives restarts
Reviewer queue	`interrupt()` blocks the call	Priority queue: EMERGENCY 60 s SLA · trap 5 min · low-conf 15 min · fallback 30 min
RAG	local Chroma in process	Hosted vector DB at 100K+ rows, shard collections by `final_category`
Trainer-log retention	every row in JSON	All hot for 30 d · overrides indefinite · 10% sample of agreement rows

Cost envelope — ~2K input / ~200 output tokens per call @ `gpt-4.1-mini`; still low enough for 10K/day intake, with vector DB + Postgres negligible at this scale.

15 · Clarification + inarticulate-intake policy

Ask one disambiguating question only when one of three signals fires (precision over recall — auto-resolution axis penalizes unnecessary clarifications):

1. **Sparse caller** — first utterance ≤6 words AND total caller speech ≤24 words (conjunction separates back-fillers from genuinely vague callers).
2. **Generic opening** — `"there's an issue with X"`, `"something's wrong with Y"`, `"the X is being weird / off"` — caller used the category label as the whole symptom.
3. **Floor self-correction** — caller mentions ≥2 distinct floor numbers ("Floor 9 — I meant 12") — typical `location_conflict` pattern.

Inarticulate / incomplete intake — when transcripts are very short, garbled, or non-English, or when no location is recoverable: (a) the question above runs first, (b) if still incomplete after one round, `needs_human_review=True` and `dispatched_vendor_id=None` (unroutable handoff), (c) `language` field flows into the trainer log so language-specific reviewer routing is possible at scale.

16 · Voice intake end-to-end (bonus) — architecture + demo plan

Architecture (zero changes to `classify()` — same callable for both modes)

```
mic → Deepgram (STT, streaming)
    → accumulate {speaker, text} turns
    → classify(turns, caller_phone)
    → ElevenLabs (TTS reads call_summary)
```

`voice/VoiceSession` (PR #81) — Deepgram Nova-2 STT + ElevenLabs Turbo v2.5 TTS. Pure I/O module; thin FastAPI adapter wraps the same `classify`.

Demo plan today

- **Primary path:** canned-transcript buttons in the Next.js frontend (PR #78) + FastAPI/SSE backend (PR #79). Reliable on stage Wi-Fi.
- **Bonus path (if wired):** mic-on opening demo using `VoiceSession`. The pipeline downstream is byte-identical — a `tests/test_classify_unchanged.py` source-hash pin guarantees it.

17 · What's next — the open work

- **Latency** (`#27`): currently ~88 min / 1K vs <60 min AC target. Smaller extract prompt + reduce k from 8 to 5 with re-ranker — measurable on dev in one run.
- **HITL precision** (`#64` residual): 16 FP cluster on `air_quality / pipe_leak / power_outage` at MEDIUM — same-turn benign-context override candidate.
- **Risk on `edge` case-type**: 5/16 correct, weakest pocket — natural next target after HITL precision.
- **Vendor city-coverage misses** (`#66`): 6 rows with strict city filter; partial-match fallback w/ adjacent-city graph is the open lever.
- **Voice integration**: PR #81 wired into FastAPI for live mic demo (currently canned only).
- **Test-set composite**: re-eval running concurrently — locked submission state pending before lockdown.

18 · Q&A

Composite: `93.28 / 100` on dev · final `predictions.json` has 800 validated test rows · `0 false-911` across all measured runs · 9 nodes · 36 subcategories · 10K-ticket RAG.

Sources (all in this repo)

- `docs/design_doc.md` — 11 sections, all derivations + error analysis
- `eval_runs/README.md` — full per-run history with deltas
- `agent/data/derived/` — base-risk table · HITL policy · vendor-type map
- `evaluation/scoring.py::AXIS_WEIGHTS` — rubric ground truth
- `backend/demo/canned.py` — the three demo transcripts

> "Auto-resolve the routine. Escalate when judgment is required. Never autonomously dispatch 911 unless five gates agree."

Demo Script

> *Three canned demos from `backend/demo/canned.py` — pre-loaded as buttons in the Next.js frontend (PR #78) calling the FastAPI/SSE backend (PR #79). Pipeline stages light up in real time as the agent runs. If voice (PR #81) is wired by demo time, prepend a **Demo 0** spoken-mic version of Demo 1; otherwise the canned-button path is the primary demo.*

Demo 0 (only if voice/ is wired into backend by demo time) — ~30 s

Setup. Switch frontend to "voice" mode. Press-to-talk.

Speaker says into the mic:

> *"Hi, this is Maria from Suite 408 at Pacific Ridge Medical Plaza — our AC has been out since 8am and it's getting really warm in here."*

Expected behavior:

1. Deepgram streams partial transcripts into the turns panel.
2. `end_utterance()` finalizes; `classify()` runs the same way it would in batch.
3. ElevenLabs reads back the agent's `call_summary` over the speakers.
4. UI shows the same per-stage SSE events as the canned demos below.

Speaker says, while pipeline runs:

> *"Same callable as the batch grader — no fork. Voice is a thin I/O adapter; everything downstream is byte-identical to the scored submission."*

Demo 1 — Routine: AC failure, auto-resolve (~30 s)

Transcript ID: `EVAL-0976` — "AC failure, LOW, routine auto-resolve."

Speaker says, while pointing at the demo UI:

> *"First demo — routine call. AC out at a medical plaza. Watch the pipeline."*

Click: `EVAL-0976` button in the demo frontend.

Expected behavior, stage by stage (frontend SSE shows each lighting green):

STAGE	OUTPUT	WALL TIME
extract	`problem_summary="AC not cooling"` , `urgency_cues=[]` , building/floor populated	~1–2 s
classify	`category=HVAC` , `subcategory=no_cooling` , `conf > 0.8`	~2 s
reconcile_location	transcript wins; building + address + floor populated	<0.1 s
assign_risk	`risk=LOW` (base prior, no modifiers)	<0.1 s
validator	`should_pause=False` — no pause-rule fires; **auto-resolve**	<0.1 s
vendor_select	qualifies on `HVAC` , city match, SLA cap 480 min → top-rated vendor returned	<0.5 s
clarify	sparse-caller check fails (caller gave detail) → `needs_clarification=False`	<0.1 s

Demo 2 — Over-escalation trap: "burnt toast" (~60 s)

Transcript ID: `EVAL-0788` — "smoke alarm — just burnt toast", over-escalation trap.

Speaker says:

> *"This is the trap — the −5-point rubric penalty. A naive system reaches 'smoke alarm' and dispatches 911. Watch what ours does."*

Click: `EVAL-0788` button.

Expected behavior:

STAGE	OUTPUT
extract	`urgency_cues=["smoke alarm going off"]`, `negation_phrases=["no actual fire", "just burnt the toast"]`
classify	`category=JANITORIAL`, `subcategory=waste_odor`, `conf > 0.7` — RAG returns the seeded `[QA-RECLASSIFIED from intake=fire_smoke]` examples; boundary rule "burnt food → waste_odor, not fire_smoke" fires
assign_risk	base = LOW for `waste_odor`; "smoke" cue would add +1, but **same-turn negation guard cancels it** → final `risk=LOW`
validator	`should_pause=False` — `waste_odor` is not on trap-prone list at LOW; **auto-resolve, no 911**

Demo 3 — True emergency: visible flames (~30 s)

Transcript ID: `EVAL-0076` — "visible smoke + flames in electrical room, true EMERGENCY."

Speaker says:

> *"And the other direction — the actual emergency. Same pipeline, opposite outcome."*

Click: `EVAL-0076` button.

Expected behavior:

STAGE	OUTPUT
extract	`urgency_cues=["smoke", "fire", "flames"]`, `negation_phrases=[]`
classify	`category=LIFE_SAFETY`, `subcategory=fire_smoke`, `conf > 0.9`
assign_risk	base EMERGENCY for `fire_smoke` + hard urgency cue → cap at **EMERGENCY**
validator	all five conditions hold → `dispatched_emergency_services=True` AND `needs_human_review=True` (notify async). Pipeline does **not** wait.
vendor_select	emergency electrician with `available_24_7=True`, SLA ≤30 min

Speaker says:

Verification cheat-sheet (for the speaker)

WHAT WE PROVED	SLIDE	DEMO
Auto-resolve routine	1, 6, 11	1
Over-escalation guard	4, 8, 12	2
Safety mechanism (5-gate 911)	12	3
HITL design (interrupt/resume)	7	2 (override beat)
Trainer-log replay	10	2 (override beat)
Clarification + inarticulate intake	15	(covered narratively)
Single callable contract (voice + batch)	16	0

Time budget (5 min target). 18 slides total; not every slide needs equal time. Plan:

BLOCK	SLIDES	TIME
Framing	1–2	30 s
Derived policy	3–5	50 s
Risk + HITL + RAG + vendor	6–9	70 s
Trainer log + scores + safety	10–12	40 s
Scale + clarify + voice	13–16	40 s
Next + Q&A	17–18	20 s
Live demo (1+2+3)	—	**2 min**